

茅台酒溯源区块链系统

学号: 2354100

姓名: 郝哲逸

课程: 信息安全

设计题目: 区块链设计

一、设计思路

1.1 问题背景

茅台酒作为高端白酒，市场上存在大量假冒伪劣产品。传统溯源系统存在以下问题：

- **数据易篡改**：中心化数据库的数据可被管理员修改
- **信息不透明**：消费者无法验证数据的真实性
- **责任难追溯**：出现问题时难以定位责任环节

区块链技术具有去中心化、不可篡改、透明可追溯等特性，非常适合用于产品溯源场景。

1.2 需求分析

本系统需要实现以下核心功能：

功能模块	具体需求
产品注册	生产厂家注册新产品，记录产品基本信息
运输追踪	记录产品在各节点间的流转过程
溯源查询	消费者可查询产品的完整历史记录
节点管理	管理员授权供应链各环节的参与节点

1.3 区块链技术选型

选择以太坊（Ethereum）作为区块链平台：

特性	优势
智能合约	支持 Solidity 编程，可实现复杂业务逻辑
开发工具	Hardhat、Ganache 等工具成熟，便于开发和测试
本地测试	可快速搭建私有链进行测试，无需消耗真实代币
社区支持	文档完善，社区活跃，问题易于解决

1.4 系统设计原则

1. 数据不可篡改

- 所有产品信息和运输记录存储在区块链上
- 一旦写入区块，数据永久保存且无法修改

2. 权限分级管理

- 管理员：负责节点授权和系统管理
- 授权节点：可注册产品和记录运输
- 普通用户：只能查询产品信息

3. 全程可追溯

- 每次运输都生成一条永久记录
- 形成完整的产品生命周期链条

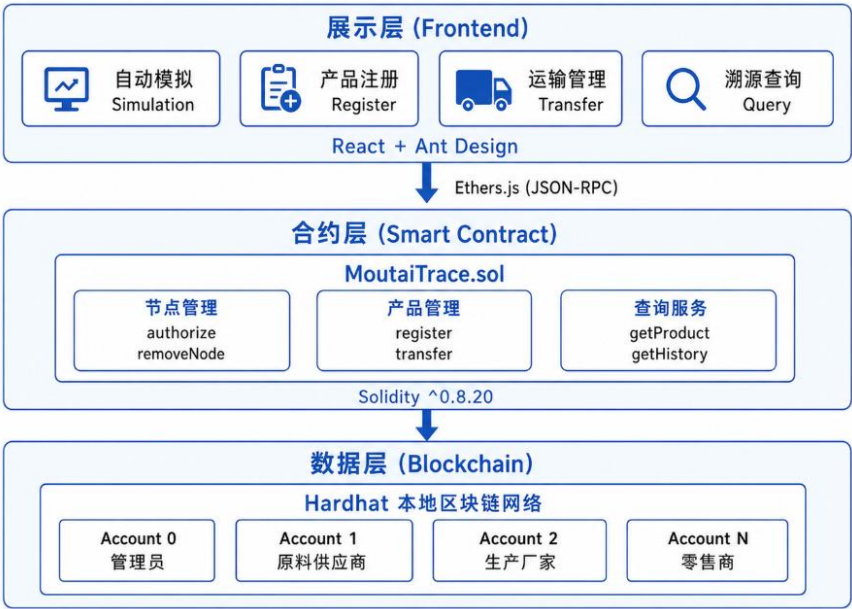
4. 操作便捷

- 提供一键模拟功能，快速生成演示数据
- 界面简洁直观，降低使用门槛

二、系统架构

2.1 整体架构设计

系统采用经典的三层架构：展示层、合约层、数据层。



2.2 技术栈详解

层级	技术	版本	选型理由
区块链平台	Ethereum	-	成熟的智能合约平台，生态完善
开发框架	Hardhat	^2.22.0	现代化开发框架，支持编译、测试、部署
智能合约	Solidity	^0.8.20	以太坊官方合约语言，安全可靠
前端框架	React	18.x	组件化开发，生态丰富
构建工具	Vite	5.x	快速热更新，开发体验好
UI 组件库	Ant Design	5.x	企业级组件，界面美观
Web3 库	Ethers.js	6.x	轻量级，API 友好

2.3 智能合约设计

2.3.1 数据模型设计

系统设计了三个核心数据结构：

(1) 节点信息结构

用于存储供应链参与方的详细信息：

```
STRUCT NodeInfo {
    STRING NAME;           // 节点名称 (如: 茅台酒厂 A)
    STRING PHONE;          // 联系电话
    STRING CATEGORY;       // 类别: 原料供给/产品生产/批发销售/零售
    STRING LOCATION;       // 实体地址
    STRING DESCRIPTION;    // 节点描述
    BOOL ISAUTHORIZED;     // 授权状态
    BOOL EXISTS;           // 记录是否存在
}
```

(2) 产品信息结构

用于存储茅台酒产品的基本信息：

```
STRUCT Product {
    UINT256 ID;            // 产品唯一标识符
    STRING NAME;           // 产品名称 (如: 飞天茅台)
    STRING BATCHNO;        // 批次号 (如: MT-2024-001)
    UINT256 PRODUCTIONDATE; // 生产日期 (UNIX 时间戳)
    ADDRESS MANUFACTURER;  // 生产厂家账户地址
    ADDRESS CURRENTOWNER;  // 当前持有者账户地址
    UINT8 CURRENTSTAGE;    // 当前所处阶段
    BOOL EXISTS;           // 产品是否存在
}
```

(3) 运输记录结构

用于记录产品在节点间的流转过程：

```
STRUCT TransferRecord {
    UINT256 PRODUCTID;     // 关联的产品 ID
    ADDRESS FROM;          // 发送方账户地址
    ADDRESS TO;            // 接收方账户地址
    STRING FROMLOCATION;     // 发货地点
    STRING TOLOCATION;       // 收货地点
    UINT256 TIMESTAMP;     // 运输时间戳
    STRING REMARK;         // 备注信息
    STRING VERIFIER;       // 验证员姓名
}
```

2.3.2 产品生命周期设计

产品从生产到销售经历四个阶段，阶段只能单向递增：

阶段	阶段值	参与者	可执行操作
原料阶段	0	原料供应商	提供原料
生产阶段	1	生产厂家	注册产品、发货

阶段	阶段值	参与者	可执行操作
物流阶段	2	批发商	接收货物、配送
零售阶段	3	零售商	接收货物、销售

2.3.3 核心功能函数设计

(1) 节点授权函数

```
FUNCTION AUTHORIZENODEWITHINFO(  
    ADDRESS _NODE,      // 待授权的账户地址  
    STRING MEMORY _NAME, // 节点名称  
    STRING MEMORY _PHONE, // 联系电话  
    STRING MEMORY _CATEGORY, // 节点类别  
    STRING MEMORY _LOCATION, // 实体地址  
    STRING MEMORY _DESCRIPTION  
)  
EXTERNAL ONLYADMIN {  
    // 将节点信息存入映射  
    NODEINFO[_NODE] = NODEINFO({  
        NAME: _NAME,  
        PHONE: _PHONE,  
        CATEGORY: _CATEGORY,  
        LOCATION: _LOCATION,  
        DESCRIPTION: _DESCRIPTION,  
        ISAUTHORIZED: TRUE,  
        EXISTS: TRUE  
    });  
    // 标记为授权节点  
    AUTHORIZEDNODES[_NODE] = TRUE;  
    // 触发事件，供前端监听  
    EMIT NODEAUTHORIZED(_NODE, _NAME, _CATEGORY);  
}
```

设计要点：

- 只有管理员可以调用（onlyAdmin 修饰器）
- 存储完整的节点信息，便于溯源时展示
- 触发事件，前端可监听授权成功状态

(2) 产品注册函数

```
FUNCTION REGISTERPRODUCT(  
    STRING MEMORY _NAME,      // 产品名称  
    STRING MEMORY _BATCHNo,   // 批次号  
    UINT256 _PRODUCTIONDATE   // 生产日期时间戳  
)  
EXTERNAL ONLYAUTHORIZEDNODE RETURNS (UINT256) {  
    // 产品ID 自增，保证唯一性  
    PRODUCTCOUNT++;  
    UINT256 NEWPRODUCTID = PRODUCTCOUNT;  
  
    // 创建产品记录  
    PRODUCTS[NEWPRODUCTID] = PRODUCT({  
        ID: NEWPRODUCTID,  
        NAME: _NAME,  
        BATCHNo: _BATCHNo,  
        PRODUCTIONDATE: _PRODUCTIONDATE,  
        MANUFACTURER: MSG.SENDER, // 调用者即为生产厂家  
        CURRENTOWNER: MSG.SENDER, // 当前持有者为生产厂家  
        CURRENTSTAGE: 1,           // 初始阶段为生产阶段  
        EXISTS: TRUE  
    });  
  
    // 记录产品ID，便于遍历查询  
    ALLPRODUCTIDS.PUSH(NEWPRODUCTID);  
}
```

```

    EMIT PRODUCTREGISTERED(NEWPRODUCTID, _NAME, _BATCHNO, MSG.SENDER, _PRODUCTIONDATE);
    RETURN NEWPRODUCTID;
}

```

设计要点：

- 只有授权节点可以调用 (`onlyAuthorizedNode` 修饰器)
- 自动设置调用者为生产者和持有者
- 产品 ID 由合约自动生成，避免冲突

(3) 运输记录函数

```

FUNCTION RECORDTRANSFERWITHVERIFIER(
    UINT256 _PRODUCTID,    // 产品 ID
    ADDRESS _TO,            // 接收方地址
    STRING MEMORY _FROMLOCATION, // 发货地点
    STRING MEMORY _TOLOCATION, // 收货地点
    STRING MEMORY _REMARK,   // 备注信息
    STRING MEMORY _VERIFIER  // 验证员姓名
) EXTERNAL PRODUCTEXISTS( _PRODUCTID) ONLYAUTHORIZEDNODE {
    PRODUCT STORAGE PRODUCT = PRODUCTS[_PRODUCTID];

    // 权限检查：只有当前持有者才能转移
    REQUIRE(PRODUCT.CURRENTOWNER == MSG.SENDER, "NOT OWNER");
    // 接收方必须是授权节点
    REQUIRE(AUTHORIZEDNODES[_TO], "NOT AUTHORIZED");
    // 阶段检查：不能超过零售阶段
    REQUIRE(PRODUCT.CURRENTSTAGE < 3, "INVALID STAGE");

    // 阶段递增
    UINT8 NEXTSTAGE = PRODUCT.CURRENTSTAGE + 1;
    TRANSFERCOUNT++;

    // 创建运输记录
    TRANSFERRECORD MEMORY RECORD = TRANSFERRECORD({
        PRODUCTID: _PRODUCTID,
        FROM: MSG.SENDER,
        TO: _TO,
        FROMLOCATION: _FROMLOCATION,
        TOLOCATION: _TOLOCATION,
        TIMESTAMP: BLOCK.TIMESTAMP, // 使用区块时间戳
        REMARK: _REMARK,
        VERIFIER: _VERIFIER
    });

    // 存储运输历史
    PRODUCTHISTORY[_PRODUCTID].PUSH(RECORD);

    // 更新产品状态
    PRODUCT.CURRENTOWNER = _TO;
    PRODUCT.CURRENTSTAGE = NEXTSTAGE;

    EMIT TRANSFERRECORDED(_PRODUCTID, MSG.SENDER, _TO,
        _FROMLOCATION, _TOLOCATION, BLOCK.TIMESTAMP, _REMARK, _VERIFIER);
}

```

设计要点：

- 三重权限验证：持有者、接收方授权、阶段有效性
- 使用区块链时间戳，不可伪造
- 运输历史永久保存在数组中

(4) 溯源查询函数

```

FUNCTION GETPRODUCTHISTORY(UINT256 _PRODUCTID)
    EXTERNAL VIEW PRODUCTEXISTS( _PRODUCTID)

```

```
    RETURNS (TRANSFERRECORD[] MEMORY)
  {
    RETURN PRODUCTHISTORY[_productId];
  }
}
```

设计要点：

- 任何人都可以查询，无需权限
- 返回完整的运输历史数组
- 数据来自区块链，保证真实性

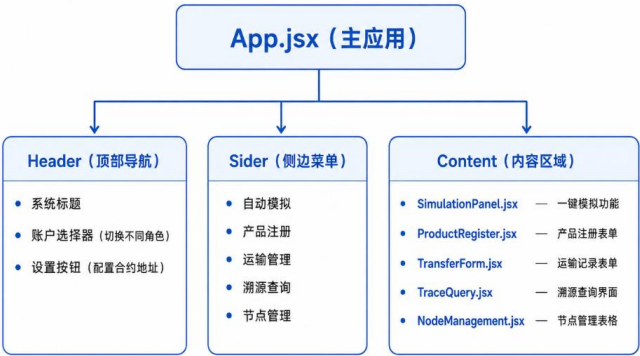
2.4 权限控制机制

系统采用三级权限控制：



2.5 前端架构设计

2.5.1 组件结构



2.5.2 Web3 交互流程

前端通过 Ethers.js 与区块链交互：

```
// 1. 连接区块链节点
const provider = new ethers.JsonRpcProvider('http://127.0.0.1:8545');

// 2. 获取账户列表
const accounts = await provider.listAccounts();
```

```
// 3. 创建合约实例
CONST CONTRACT = NEW ETHERS.CONTRACT(
  CONTRACTADDRESS, // 合约地址
  CONTRACTABI,      // 合约ABI
  SIGNER            // 签名者 (账户)
);

// 4. 调用合约方法
CONST TX = AWAIT CONTRACT.REGISTERPRODUCT("飞天茅台", "MT-001", TIMESTAMP);
AWAIT TX.WAIT(); // 等待交易确认

// 5. 查询数据
CONST PRODUCT = AWAIT CONTRACT.GETPRODUCT(1);
CONST HISTORY = AWAIT CONTRACT.GETPRODUCTHISTORY(1);
```

三、程序界面说明

3.1 系统主界面



系统采用经典的后台管理布局：

区域	说明
顶部导航栏	显示系统标题、当前账户选择器、设置按钮
左侧菜单	五个功能模块的导航入口
内容区域	当前选中模块的操作界面

3.2 自动模拟模块



功能说明：一键授权 12 个供应链节点（2 个原料供应商、3 个生产商、2 个批发商、5 个零售商）、随机生成 3-5 个茅台酒产品、自动模拟每个产品从生产到零售的完整流程、显示实时进度条和当前操作

3.3 产品注册模块

+ 产品注册

当前账户: 0xf39f...2266

* 产品名称

例如：飞天茅台

* 批次号

例如：BATCH-2024-001

* 生产日期

2026-05-03

+ 注册产品

注册成功

产品ID: 11

产品名称: 飞天茅台

批次号: BATCH-2024-001

生产日期: 2026-05-03

操作流程：

- 1. 选择账户切换到生产厂家
- 2. 填写产品名称（如：飞天茅台）
- 3. 输入批次号（如：MT-2024-001）
- 4. 选择生产日期
- 5. 点击"注册产品"按钮
- 6. 显示注册成功的产品 ID

3.4 运输管理模块

与 运输管理

当前账户: 0xf39f...2266

* 选择产品

飞天茅台 (ID: 12) - 生产阶段

产品名称: 飞天茅台

批次号: BATCH-2024-001

当前阶段: 生产

原料

2 生产

3 物流

4 零售

* 目标节点

批发中心A (0x976E...0aa9)

* 发货地点

上海

* 收货地点

贵州

验证员

刘军李

备注

运输备注信息

与 记录运输

操作流程：

- 1. 选择要运输的产品（显示当前阶段）
- 2. 选择目标节点（只显示已授权节点）

- 3. 填写发货地点和收货地点
- 4. 可选择验证员
- 5. 填写备注信息
- 6. 点击"记录运输"按钮

3.5 溯源查询模块

溯源查询

查询成功!

1

查询

产品信息

零售阶段

产品ID	1	产品名称	华茅酒
批次号	FY-2026-4272	生产日期	2025/12/31
生产厂家	0x9965...A4dc	当前持有者	0xa0Ee...9720

产品生命周期

原料

生产

物流

零售

运输轨迹

时间: 2026/5/3 18:35:52

发货地点: 贵州遵义市

发送方: 茅台酒厂C(0x9965...A4dc)

验证员: 吴晓明

收货地点: 北京物流中心

接收方: 批发中心A(0x976E...0aa9)

备注: 产品出厂, 质量检验合格

时间: 2026/5/3 18:35:53

发货地点: 广州物流中心

发送方: 批发中心A(0x976E...0aa9)

验证员: 赵晓梅

收货地点: 上海专卖店

接收方: 零售店B(0xa0Ee...9720)

备注: 批发销售, 运输完成

查询结果展示：产品基本信息（ID、名称、批次号、生产日期、当前阶段）、产品生命周期进度条、运输历史时间线（时间、地点、发送方、接收方、验证员）

3.6 节点管理模块

节点管理

节点地址

0x139Fde5...fb92266

0x70997970...17dc79C8

0x3C44CdDd...FA42938C

0x90F79bf6...1E93b906

0x15d34AAf...0a2C6A65

0x9965507D...1980A4dc

0x976EA740...0C3a0aa9

0x14dC7996...193d9955

0x23618e81...f5821E8f

0xa0Ee7A14...20a79720

0xBcd4042D...3b954096

节点名称

例如: 茅台酒厂A、物流中心B

联系电话

例如: 13800138000

节点类别

产品生产

地址

例如: 贵州铜仁怀市茅台镇

描述

节点描述信息

授权

操作

详情

详情

详情

详情

详情

详情

详情

详情

详情

详情

移除

移除

移除

移除

移除

移除

移除

移除

移除

移除

节点列表信息：节点地址、节点名称、节点类别（带颜色标签）、授权状态、操作按钮（详情、移除）

授权表单字段：节点地址（下拉选择）、节点名称、联系电话、节点类别、地址、描述

3.7 系统设置

系统设置

合约地址

0x5FbDB2315678afecb367f032d93F642f64180aa3

连接

连接状态

✓ 已连接到区块链节点

✓ 已连接合约

✓ 当前账户是管理员

可用账户

账户 0: 0xf39Fd6e5...fb922266

账户 1: 0x70997970...17dc79C8

账户 2: 0x3C44CdDd...FA4293BC

账户 3: 0x90F79hf6...1F93h906

设置内容：合约地址输入框、连接状态显示（区块链连接、合约连接、管理员身份）、可用账户列表

四、程序代码

4.1 项目目录结构

moutai-trace/

contracts/

MoutaiTrace.sol

scripts/

deploy.js

test/

MoutaiTrace.test.js

frontend/

src/

components/

SimulationPanel.jsx

ProductRegister.jsx

TransferForm.jsx

TraceQuery.jsx

NodeManagement.jsx

hooks/

useContract.js

contracts/

MoutaiTrace.json

App.jsx

main.jsx

package.json

vite.config.js

hardhat.config.js

package.json

start.bat

start.sh

README.md

智能合约目录

茅台酒溯源主合约

脚本目录

合约部署脚本

测试目录

合约单元测试 (19 个用例)

前端应用目录

React 组件

自动模拟

产品注册

运输管理

溯源查询

节点管理

合约交互 Hook

合约 ABI

主应用

入口文件

Hardhat 配置

项目依赖

Windows 启动脚本

Linux 启动脚本

项目说明

本项目采用前后端分离与智能合约独立管理的目录结构。contracts 目录用于存放 Solidity 智能合约文件，是系统业务逻辑的核心；scripts 目录用于存放合约部署脚本，方便将合约部署到 Hardhat 本地区块链网络；test 目录用于存放合约测试文件，保证主要功能在运行前能够被验证。前端部分集中在 frontend 目录下，其中 components 用于存放各功能模块组件，hooks 用于封装合约交互逻辑，contracts 用于保存合约 ABI 文件，便于前端调用链上方法。这样的目录划分层次清晰，便于开发、调试和后期维护。

4.2 智能合约核心代码

4.2.1 状态变量与数据结构

状态变量用于记录系统运行过程中的核心数据，例如管理员地址、产品总数、运输记录总数、产品映射、运输历史映射、节点信息映射和授权节点映射等。通过 mapping 结构，可以根据产品 ID 或账户地址快速查询对应数据；通过 allProductIds 数组，可以保存所有产品编号，便于前端进行列表展示和批量查询。该部分代码为整个合约提供了基础数据存储结构，是后续功能函数运行的前提。

```
// SPDX-License-Identifier: MIT
PRAGMA SOLIDITY ^0.8.20;

CONTRACT MOUTAITrace {
    // ===== 状态变量 =====

    ADDRESS public ADMIN;          // 管理员地址
    uint256 public PRODUCTCOUNT;  // 产品总数
    uint256 public TRANSFERCOUNT; // 运输记录总数

    // 产品 ID => 产品信息
    mapping(uint256 => Product) public PRODUCTS;

    // 产品 ID => 运输记录数组
    mapping(uint256 => TransferRecord[]) public PRODUCTHISTORY;

    // 地址 => 节点信息
    mapping(address => NodeInfo) public NODEINFO;

    // 地址 => 是否授权
    mapping(address => bool) public AUTHORIZEDNODES;

    // 所有产品 ID (用于遍历)
    uint256[] public ALLPRODUCTIDS;
}
```

4.2.2 权限控制修饰器

权限控制修饰器用于限制不同角色能够执行的操作。onlyAdmin 限制只有管理员可以进行节点授权等管理操作；onlyAuthorizedNode 限制只有被授权的供应链节点才能注册产品或记录运输；productExists 用于检查产品是否存在，避免对无效产品进行操作。通过修饰器的方式进行权限判断，可以减少重复代码，使合约逻辑更加简洁，也能提高系统的安全性和可靠性。

```
// 只有管理员可以调用
modifier onlyAdmin() {
    require(msg.sender == ADMIN, "ONLY ADMIN CAN CALL THIS");
    _;
}

// 只有授权节点可以调用
modifier onlyAuthorizedNode() {
    require(AUTHORIZEDNODES[msg.sender], "ONLY AUTHORIZED NODES");
    _;
}

// 产品必须存在
modifier productExists(uint256 productId) {
    require(PRODUCTS[_productId].exists, "PRODUCT NOT FOUND");
    _;
}
```

4.2.3 产品注册函数

产品注册函数用于将新的茅台酒产品信息写入区块链。函数首先检查产品名称和批次号是否为空，然后通过 `productCount` 自动递增生成唯一产品 ID，接着将产品名称、批次号、生产日期、生产厂家、当前持有者和当前阶段等信息保存到 `products` 映射中。由于注册者必须是授权节点，因此可以保证只有合法的供应链参与方才能创建产品记录。函数最后触发产品注册事件，便于前端监听并反馈注册结果。

```
FUNCTION REGISTERPRODUCT(  
    STRING MEMORY _NAME,  
    STRING MEMORY _BATCHNo,  
    UINT256 _PRODUCTIONDATE  
) EXTERNAL ONLYAUTHORIZEDNODE RETURNS (UINT256) {  
    // 输入验证  
    REQUIRE(BYTES(_NAME).LENGTH > 0, "NAME CANNOT BE EMPTY");  
    REQUIRE(BYTES(_BATCHNo).LENGTH > 0, "BATCH NUMBER CANNOT BE EMPTY");  
  
    // 生成新的产品 ID  
    PRODUCTCOUNT++;  
    UINT256 NEWPRODUCTID = PRODUCTCOUNT;  
  
    // 创建产品记录  
    PRODUCTS[NEWPRODUCTID] = PRODUCT({  
        ID: NEWPRODUCTID,  
        NAME: _NAME,  
        BATCHNo: _BATCHNo,  
        PRODUCTIONDATE: _PRODUCTIONDATE,  
        MANUFACTURER: MSG.SENDER,  
        CURRENTOWNER: MSG.SENDER,  
        CURRENTSTAGE: 1, // 初始为生产阶段  
        EXISTS: TRUE  
    });  
  
    ALLPRODUCTIDS.PUSH(NEWPRODUCTID);  
  
    EMIT PRODUCTREGISTERED(NEWPRODUCTID, _NAME, _BATCHNo, MSG.SENDER, _PRODUCTIONDATE);  
    RETURN NEWPRODUCTID;  
}
```

4.2.4 运输记录函数

运输记录函数用于记录产品从一个供应链节点流转 to 另一个节点的过程。函数首先检查调用者是否为当前产品持有者，再检查接收方是否为授权节点，同时判断产品是否已经到达零售阶段，避免重复或非法流转。通过这些限制，系统能够保证每一次运输记录都符合业务流程。运输记录中包含发货方、接收方、发货地点、收货地点、时间戳、备注和验证员等信息，这些数据会被永久保存到区块链中，形成完整的产品流转历史。

```
FUNCTION RECORDTRANSFERWITHVERIFIER(  
    UINT256 _PRODUCTID,  
    ADDRESS _TO,  
    STRING MEMORY _FROMLOCATION,  
    STRING MEMORY _TOLOCATION,  
    STRING MEMORY _REMARK,  
    STRING MEMORY _VERIFIER  
) EXTERNAL PRODUCTEXISTS(_PRODUCTID) ONLYAUTHORIZEDNODE {  
    PRODUCT STORAGE PRODUCT = PRODUCTS[_PRODUCTID];  
  
    // 权限验证  
    REQUIRE(PRODUCT.CURRENTOWNER == MSG.SENDER, "ONLY OWNER CAN TRANSFER");  
    REQUIRE(AUTHORIZEDNODES[_TO], "RECIPIENT NOT AUTHORIZED");  
    REQUIRE(PRODUCT.CURRENTSTAGE < 3, "PRODUCT ALREADY AT RETAIL STAGE");  
  
    // 阶段递增  
    UINT8 NEXTSTAGE = PRODUCT.CURRENTSTAGE + 1;  
    TRANSFERCOUNT++;  
  
    // 创建运输记录  
    PRODUCTHISTORY[_PRODUCTID].PUSH(TRANSFERRECORD({  
        PRODUCTID: _PRODUCTID,  
        FROM: MSG.SENDER,
```

```

    TO: _TO,
    FROMLOCATION: _FROMLOCATION,
    TOLOCATION: _TOLOCATION,
    TIMESTAMP: BLOCK.TIMESTAMP,
    REMARK: _REMARK,
    VERIFIER: _VERIFIER
  ));

  // 更新产品状态
  PRODUCT.CURRENTOWNER = _TO;
  PRODUCT.CURRENTSTAGE = NEXTSTAGE;

  EMIT TRANSFERRECORDED( _PRODUCTID, MSG.SENDER, _TO,
    _FROMLOCATION, _TOLOCATION, BLOCK.TIMESTAMP, _REMARK, _VERIFIER);
}

```

4.3 前端核心代码

4.3.1 合约交互 Hook

useContract 是前端与区块链交互的核心 Hook。该 Hook 主要负责初始化 Provider、获取测试账户、设置默认账户、创建合约实例以及保存合约地址等操作。通过将合约连接逻辑封装到 Hook 中，其他组件可以直接复用其中的账户、合约对象和连接状态，避免在多个页面中重复编写连接代码，提高了前端代码的复用性和可维护性。

```

import { useState, useEffect, useCallback } from 'react';
import { ethers } from 'ethers';
import CONTRACTABI from './CONTRACTS/MOUTAI TRACE.JSON';

export function useContract() {
  const [provider, setProvider] = useState(null);
  const [contract, setContract] = useState(null);
  const [account, setAccount] = useState(null);
  const [accounts, setAccounts] = useState([]);
  const [isConnected, setIsConnected] = useState(false);

  // 初始化区块链连接
  const initConnection = useCallback(async () => {
    try {
      // 连接本地 HARDHAT 节点
      const rpcUrl = 'http://127.0.0.1:8545';
      const newProvider = new ethers.JsonRpcProvider(rpcUrl);
      setProvider(newProvider);

      // 获取所有测试账户
      const allAccounts = await newProvider.listAccounts();
      setAccounts(allAccounts.map(acc => acc.address || acc));

      // 设置默认账户
      if (allAccounts.length > 0) {
        const signer = await newProvider.getSigner(0);
        setAccount(await signer.getAddress());
        setIsConnected(true);
      }
    } catch (err) {
      console.error('连接失败:', err);
    }
  }, []);

  // 设置合约地址
  const setContractAddress = useCallback((address) => {
    if (provider && address) {
      const newContract = new ethers.Contract(
        address,
        CONTRACTABI,
        provider.getSigner(0)
      );
      setContract(newContract);
      localStorage.setItem('contractAddress', address);
    }
  }, [provider]);

  useEffect(() => {
    initConnection();
    setContractAddress(localStorage.getItem('contractAddress'));
  }, []);
}

```

```

    INITCONNECTION();
  }, [INITCONNECTION]);

  RETURN { PROVIDER, CONTRACT, ACCOUNT, ACCOUNTS, ISCONNECTED, SETCONTRACTADDRESS };
}

```

4.3.2 自动模拟核心逻辑

自动模拟功能用于快速生成演示数据，方便展示系统完整流程。代码首先将 Hardhat 提供的测试账户划分为原料供应商、生产厂家、批发商和零售商等不同角色，然后对相关节点进行授权，再自动注册产品并模拟产品从生产厂家到批发商、再到零售商的运输过程。该功能能够减少手动录入数据的时间，使用户可以快速看到产品从创建到流转的完整溯源链条，增强系统演示效果和教学展示效果。

```

CONST RUNSIMULATION = ASYNC () => {
  // 分配账户角色
  CONST MATERIALSUPPLIERS = ACCOUNTS.SLICE(1, 3); // 原料供应商
  CONST PRODUCERS = ACCOUNTS.SLICE(3, 6); // 生产厂家
  CONST WHOLESALERS = ACCOUNTS.SLICE(6, 8); // 批发商
  CONST RETAILERS = ACCOUNTS.SLICE(8, 13); // 零售商

  // 授权节点（检查避免重复授权）
  FOR (LET I = 0; I < PRODUCERS.LENGTH; I++) {
    CONST ISAUTH = AWAIT CONTRACT.AUTHORIZEDNODES(PRODUCERS[I]);
    IF (!ISAUTH) {
      AWAIT CONTRACT.AUTHORIZENODEWITHINFO(
        PRODUCERS[I],
        `茅台酒厂${STRING.FROMCHARCODE(65 + I)}`,
        PHONE, CATEGORY, LOCATION, DESCRIPTION
      );
    }
  }

  // 获取当前产品数量，计算新产品 ID
  CONST STARTPRODUCTCOUNT = AWAIT CONTRACT.PRODUCTCOUNT();

  // 注册产品并模拟运输
  FOR (LET P = 1; P <= NUMPRODUCTS; P++) {
    // 创建生产厂家的合约实例
    CONST PRODUCERCONTRACT = NEW ETHERS.CONTRACT(
      CONTRACTADDRESS, CONTRACTABI, PRODUCERSIGNER
    );

    // 注册产品
    AWAIT PRODUCERCONTRACT.REGISTERPRODUCT(NAME, BATCHNO, DATE);

    // 运输到批发商
    AWAIT PRODUCERCONTRACT.RECORDTRANSFERWITHVERIFIER(
      PRODUCTID, WHOLESALER, FROMLOC, TOLOC, REMARK, VERIFIER
    );

    // 运输到零售商
    AWAIT WHOLESALERCONTRACT.RECORDTRANSFERWITHVERIFIER(
      PRODUCTID, RETAILER, FROMLOC, TOLOC, REMARK, VERIFIER
    );
  }
};

```